A Description of the Flexible Arm Model from *Learning Stabilizing Latent Feedback Controllers via Multi-Step Backward Losses*

1 Flexible Arm Model

We consider a Hencky-type bar-chain model of a flexible arm. The arm is a planar chain of $n_s = 20$ rigid sections of length $\ell = 0.05\,\mathrm{m}$ and mass $\bar{m} = 0.04\,\mathrm{kg}$, joined by torsional springs. Let $q \in \mathbb{R}^{n_s}$ be the relative joint angles, where $q = 0$ corresponds to the upright configuration. The section angles are $\phi = Sq$, where S is the lower-triangular matrix of ones. The state is $x = (q, \dot{q}) \in \mathbb{R}^{40}$.

Denoting the configuration-dependent inertia by $M(q) \succ 0$ and its centrifugal terms by $C(q, \dot{q})\dot{q}$, the dynamics are

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + (c_0 M(q) + c_1 K_{\text{eff}})\dot{q} + \kappa K_0 q - S^\top \big(w \odot \sin(Sq)\big) = \mathbf{1}\tau. \tag{1}$$

Here, $\tau \in \mathbb{R}$ is the physical torque applied equally at every joint, $g = 9.81\,\mathrm{m/s^2}$, $w_i = (n_s - i + \frac{1}{2})g\ell\bar{m}$ for $i = 1, \ldots, n_s$, and K_0 is diagonal with entries spaced uniformly in $[1, 1.5]$. The effective stiffness and Rayleigh damping coefficients are

$$K_{\text{eff}} = \kappa K_0 - S^\top \operatorname{diag}(w) S, \qquad (c_0, c_1) = (0.4, 0.02).$$

The stiffness scale κ lies midway between the two largest generalized eigenvalues of $(S^\top \operatorname{diag}(w) S, K_0)$. Thus, K_{eff} has exactly one negative eigenvalue, and gravity destabilizes the lowest bending mode.

The inertia is obtained from the kinetic energy of the rigid sections. Let $a_{ij} = \ell$ if $j < i$, $a_{ii} = \ell/2$, and $a_{ij} = 0$ if $j > i$. Writing $N_{jk} = \sum_i \bar{m}\, a_{ij} a_{ik}$ and $I_j = \bar{m}\ell^2/12$, we have

$$M(q) = S^\top M_\phi(\phi) S, \qquad (M_\phi(\phi))_{jk} = N_{jk}\cos(\phi_j - \phi_k) + \delta_{jk} I_j, \tag{2}$$

$$C(q, \dot{q})\dot{q} = S^\top c, \qquad c_j = \sum_k N_{jk} \sin(\phi_j - \phi_k)\dot{\phi}_k^2. \tag{3}$$

The non-linear model retains these terms and the sine dependence of gravity.

2 Discretization and Feedback

Inputs are held constant for $\Delta t = 0.05\,\mathrm{s}$. The continuous linearization of (1) at the upright equilibrium is

$$A_c = \begin{bmatrix} 0 & I \\ -M(0)^{-1} K_{\text{eff}} & -M(0)^{-1}(c_0 M(0) + c_1 K_{\text{eff}}) \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ M(0)^{-1}\mathbf{1} \end{bmatrix}.$$

Set $A = e^{A_c \Delta t}$ and $B_{\text{phys}} = \int_0^{\Delta t} e^{A_c s} B_c \, ds$. The code uses the normalized input $u = s_u \tau$, where $s_u = \|B_{\text{phys}}\|_2 \approx 2.13635$, so that $B = B_{\text{phys}}/s_u$ has norm one. The discrete linearization is $x_{t+1} = Ax_t + Bu_t$. Its only unstable eigenvalue is approximately 1.129.

The two controllers are taken from the motivating example in *Learning Stabilizing Latent Feedback Controllers via Multi-Step Backward Losses*. Each uses a learned four-dimensional latent model. Let $E_0 = J_E(0)$ be the encoder Jacobian, and let (L, H) be the linearized latent dynamics. A discrete LQR gain K is computed using $Q = I_4$ and $R = I_1$, with the sign convention $u = Kz$. The implemented feedback law is

$$u_t = \operatorname{clip}\big(KE_0 x_t, -40 s_u, 40 s_u\big),$$

which limits the physical torque to 40 N·m in either direction. The model learned with the multi-step backward loss gives the `stable` controller, and the model learned with the decoder-free forward loss gives the `unstable` controller. The corresponding spectral radii $\rho(A + BKE_0)$ are approximately 0.9549 and 1.1294, respectively, when recomputed from the supplied parameters. These values describe local behavior at the upright equilibrium. Recovery from arbitrary initial configurations is not assumed.

3 Implementation and Data

The implementation uses NumPy and SciPy. The mechanical model and its non-linear integrator are ported from latentctl/systems.py. The non-linear step uses an exponential integrator in momentum form, with 50 substeps per sampling period. The linear part is integrated exactly, and each substep uses three iterations to update the configuration-dependent terms. The feedback matrices are supplied as numerical arrays, so data generation does not require training or evaluating the neural networks.

By default, initial states are drawn from $\mathcal{N}(0, 0.01^2 I_{40})$. The same random seed gives the same initial states for both controllers. The code records the full state and applied input every 0.05 s, with no added noise. It can also simulate the linearized plant or use initial states supplied by the user. For the non-linear plant, a trajectory ends at the first sampled state where a section exceeds 90° from upright. Ground contact is not modeled. The linearized plant has no angle cutoff. Each output file records the valid trajectory lengths and termination status, with unused entries filled by `NaN`.